

ME315A_Lab02

João Vitor Expedito Silva de Oliveira RA: 294836

Tarefa

Lendo 100.000 observações por vez (se você acredita que seu computador não tem memória suficiente, utilize 10.000 observações por vez), determine o percentual de vôos por Cia. Aérea que apresentou atraso na chegada (`ARRIVAL_DELAY`) superior a 10 minutos. As companhias a serem utilizadas são: AA, DL, UA e US. A estatística de interesse deve ser calculada para cada um dos dias de 2015. Para a determinação deste percentual de atrasos, apenas verbos do pacote `dplyr` e comandos de importação do pacote `readr` podem ser utilizados. Os resultados para cada Cia. Aérea devem ser apresentados em um formato de calendário.

Observação: a atividade descrita no slide pede para ler apenas 100 registros por vez. Aqui, mudamos para 100 mil registros por vez, para que haja melhor aproveitamento do tempo em classe.

Preparando Ambiente de Trabalho

```
# Baixando os pacotes necessários para a realização da atividade  
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':
```

```
filter, lag
```

```
The following objects are masked from 'package:base':
```

```
intersect, setdiff, setequal, union
```

```
library(readr)
```

Resolução das atividades a serem desenvolvidas

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)?

R: Utilizando a probabilidade frequentista, onde `ARRIVAL_DELAY` representam todos os voos, sendo eles atrasados ou não, e `ARRIVAL_DELAY > 10` representam a todos os voos que atrasaram **mais que 10 minutos**, podemos definir isso por meio de:

$$\frac{\text{ARRIVAL_DELAY} > 10}{\text{ARRIVAL_DELAY}}$$

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
 - b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe tibble;
 - e. A função deve receber apenas dois argumentos:
 - i. `input`: o conjunto de dados (referente ao lote em questão);
 - ii. `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. **É importante** observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
# Amostra do banco de dados
amostra <- read_csv("flights.csv", n_max = 5)
```

```
Rows: 5 Columns: 31
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (10): AIRLINE, TAIL_NUMBER, ORIGIN_AIRPORT, DESTINATION_AIRPORT, SCHEDUL...
```

```
dbl (15): YEAR, MONTH, DAY, DAY_OF_WEEK, FLIGHT_NUMBER, DEPARTURE_DELAY, TAX...
```

```
lg1 (6): CANCELLATION_REASON, AIR_SYSTEM_DELAY, SECURITY_DELAY, AIRLINE_DEL...
```

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
names(amostra)
```

```
[1] "YEAR"           "MONTH"           "DAY"
[4] "DAY_OF_WEEK"    "AIRLINE"          "FLIGHT_NUMBER"
[7] "TAIL_NUMBER"    "ORIGIN_AIRPORT"   "DESTINATION_AIRPORT"
[10] "SCHEDULED_DEPARTURE" "DEPARTURE_TIME"   "DEPARTURE_DELAY"
[13] "TAXI_OUT"        "WHEELS_OFF"       "SCHEDULED_TIME"
[16] "ELAPSED_TIME"    "AIR_TIME"         "DISTANCE"
[19] "WHEELS_ON"       "TAXI_IN"          "SCHEDULED_ARRIVAL"
[22] "ARRIVAL_TIME"    "ARRIVAL_DELAY"    "DIVERTED"
[25] "CANCELLED"       "CANCELLATION_REASON" "AIR_SYSTEM_DELAY"
[28] "SECURITY_DELAY"  "AIRLINE_DELAY"    "LATE_AIRCRAFT_DELAY"
[31] "WEATHER_DELAY"
```

```
getStats <- function(input, pos){
  input %>%
    filter(AIRLINE %in% c("AA","DL","UA","US")) %>% # Código referente ao Item 'a'
    filter(!is.na(ARRIVAL_DELAY)) %>% # Código referente ao Item 'b'
    group_by(YEAR, MONTH, DAY, AIRLINE) %>% # Código referente ao Item 'c'
    summarise(
      total = n(),
      atrasados = sum(ARRIVAL_DELAY > 10, na.rm = TRUE),
      .groups = "drop"
    ) #Código referente ao Item 'd'
}
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

a. Configure o tamanho do lote (chunk) para 100 mil registros;

b. Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;

c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```

banco_de_dados <- readr::read_csv_chunked(
  file = "flights.csv",
  chunk_size = 10000, # Coloquei apenas 10 mil, pois acima disso meu computador não consegue
  callback = DataFrameCallback$new(getStats),
  col_types = cols_only(
    YEAR = col_integer(),
    MONTH = col_integer(),
    DAY = col_integer(),
    AIRLINE = col_character(),
    ARRIVAL_DELAY = col_double()
  )
)

```

4. Crie uma função chamada `computeStats` que:

- a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
- b. Retorne as informações em um `tibble` contendo apenas as seguintes colunas:
 - i. Cia: sigla da companhia aérea;
 - ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
 - iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$.

```

computeStats <- function(dados){
  dados %>%
    group_by(YEAR, MONTH, DAY, AIRLINE) %>%
    summarise(
      total = sum(total),
      atrasados = sum(atrasados),
      .groups = "drop"
    ) %>%
    mutate(
      Data = as.Date(paste(YEAR, MONTH, DAY, sep = "-")),
      Perc = atrasados / total
    ) %>%
    select(Cia = AIRLINE, Data, Perc)
}

resultado_final <- computeStats(banco_de_dados)
head(resultado_final)

```

```
# A tibble: 6 x 3
  Cia    Data      Perc
  <chr> <date>    <dbl>
1 AA    2015-01-01 0.373
2 DL    2015-01-01 0.0847
3 UA    2015-01-01 0.289
4 US    2015-01-01 0.196
5 AA    2015-01-02 0.430
6 DL    2015-01-02 0.180
```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

a. Instale e carregue os pacotes `ggcal` e `ggplot2`.

b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.

c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:

i. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.

ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.

iii. Retornar para o usuário a base do calendário criada acima.

d. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.

e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Titulo")`.

```
# Baixando pacotes
if (!require(remotes)) install.packages("remotes")
```

Loading required package: remotes

```
remotes::install_github("jayjacobs/ggcal")
```

Skipping install of 'ggcal' from a github remote, the SHA1 (ab1a85ad) has not changed since .
Use `force = TRUE` to force installation

```
library(ggcal)
library(ggplot2)

# Definindo escala de cores
pal <- scale_fill_gradient(low = '#4575b4',high = '#d73027')

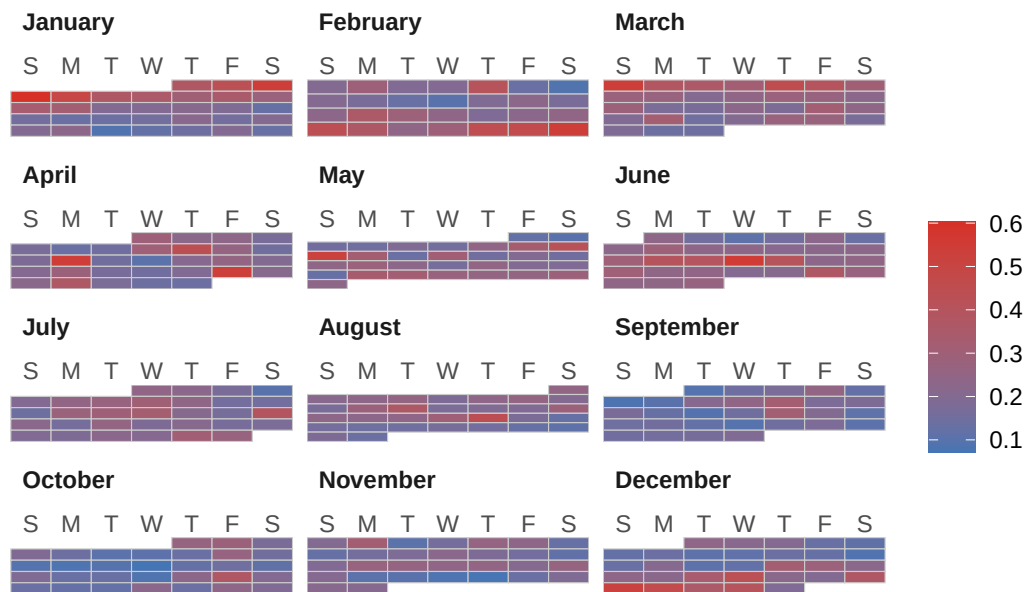
# Criação da função
baseCalendario <- function(stats, cia) {
  subconjunto <- stats %>%
    filter(Cia == cia)

  ggcal(subconjunto$Data, subconjunto$Perc)
}

# Execução em cada cia aerea
cAA <- baseCalendario(resultado_final, "AA")
cDL <- baseCalendario(resultado_final, "DL")
cUA <- baseCalendario(resultado_final, "UA")
cUS <- baseCalendario(resultado_final, "US")

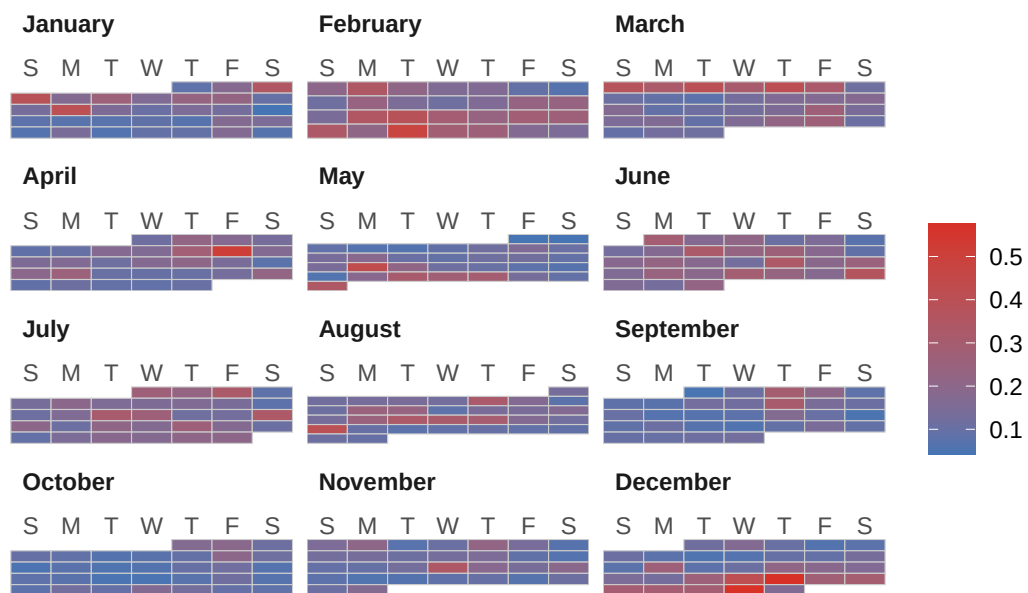
# Mapa de calor
cAA + pal + ggtitle("Percentual de Atrasos - American Airlines (AA)")
```

Percentual de Atrasos – American Airlines (AA)



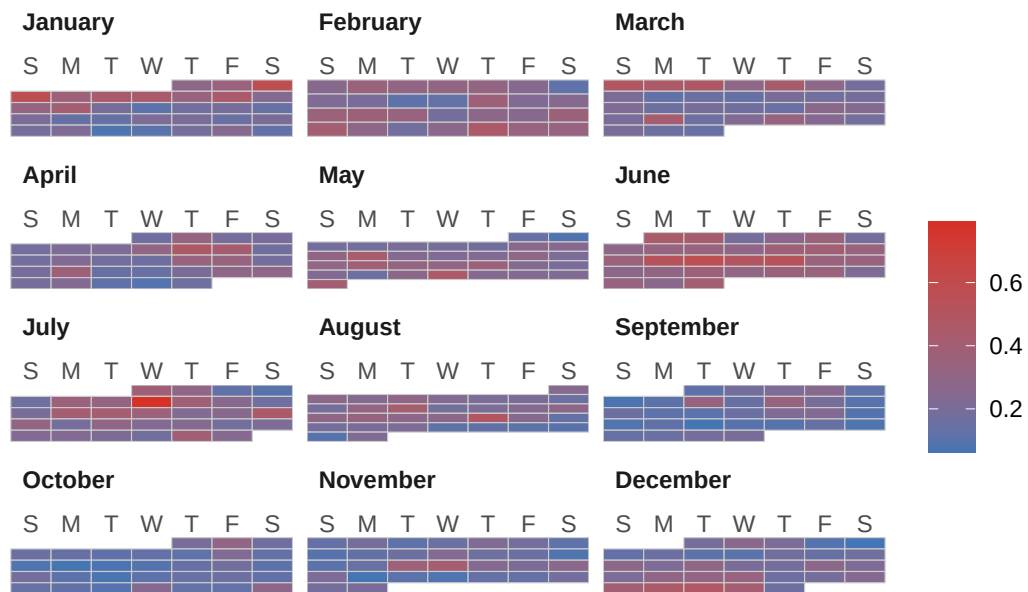
```
cDL + pal + ggtitle("Percentual de Atrasos - Delta Air Lines (DL)")
```

Percentual de Atrasos – Delta Air Lines (DL)



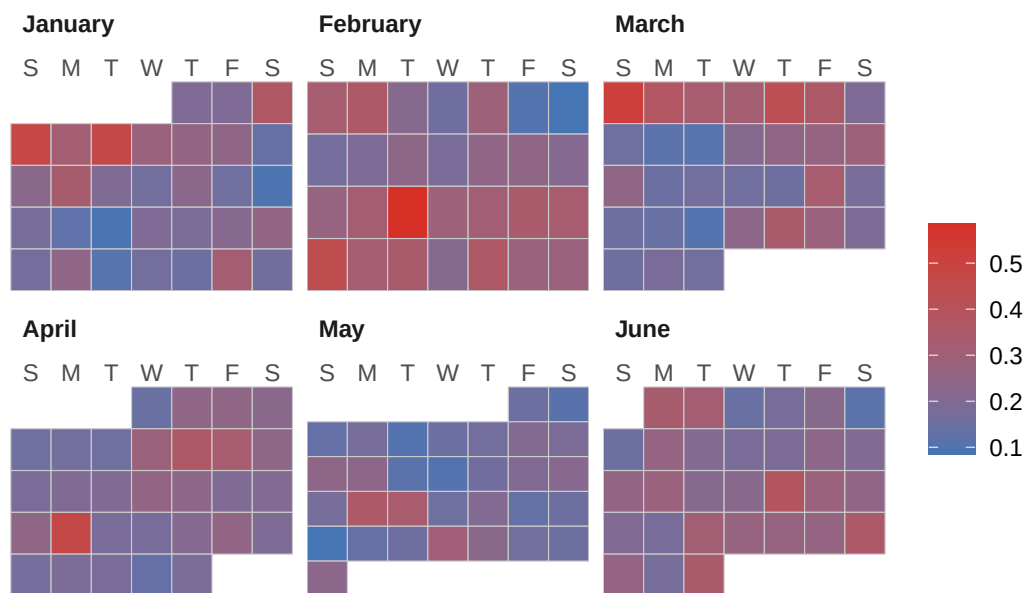
```
cUA + pal + ggtitle("Percentual de Atrasos - United Airlines (UA)")
```

Percentual de Atrasos – United Airlines (UA)



```
cUS + pal + ggtitle("Percentual de Atrasos - US Airways (US)")
```

Percentual de Atrasos – US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

[1] "2026-08-24 22:40:21 -03"